Connecting & Shaping the Data

AdventureWorks | My Power BI Learning Journey

This is the first part of the learning journal for my second business analysis portfolio project. I started with CSV files and prepared them in Power Query before moving on to data modeling and analysis. I completed this project while following a Maven Analytics course.

Data sources: 10 core CSV files become 8 tables

The core inputs comprise three yearly sales files and seven lookup/returns files. One additional file, Product Category Sales (Unpivot Demo).csv, contains 20 rows for a separate exercise and is excluded from the eight-table inventory below.

CSV / destination table	Original CSV rows	Output rows / notes
Calendar Lookup	912	912
Customer Lookup	18,154	Unconfirmed; my initial note states 18,184
Product Categories Lookup	4	4
Product Lookup	293	293
Product Subcategories Lookup	37	37
Returns Data	1,809	1,809
Sales Data 2020 / 2021 / 2022	2,630 / 23,935 / 29,481	56,046 combined
Territory Lookup	10	10

Original counts were calculated from the supplied CSV records, excluding headers. Output counts other than Customer follow my project notes; the Sales total also reconciles with the sum of the source records.

Customer reconciliation: the source contains 18,154 rows and 18,152 distinct CustomerKey values. My initial output count of 18,184 needs to be checked against the final query. Removing only the two excess key occurrences, with no other transformations, would produce 18,152 rows.

Getting to Know Power Query

I connected the files through **Get data > Text/CSV > Transform Data**. In Power Query, I inspected the preview, reviewed column quality, and recorded transformations before loading the data into the model.

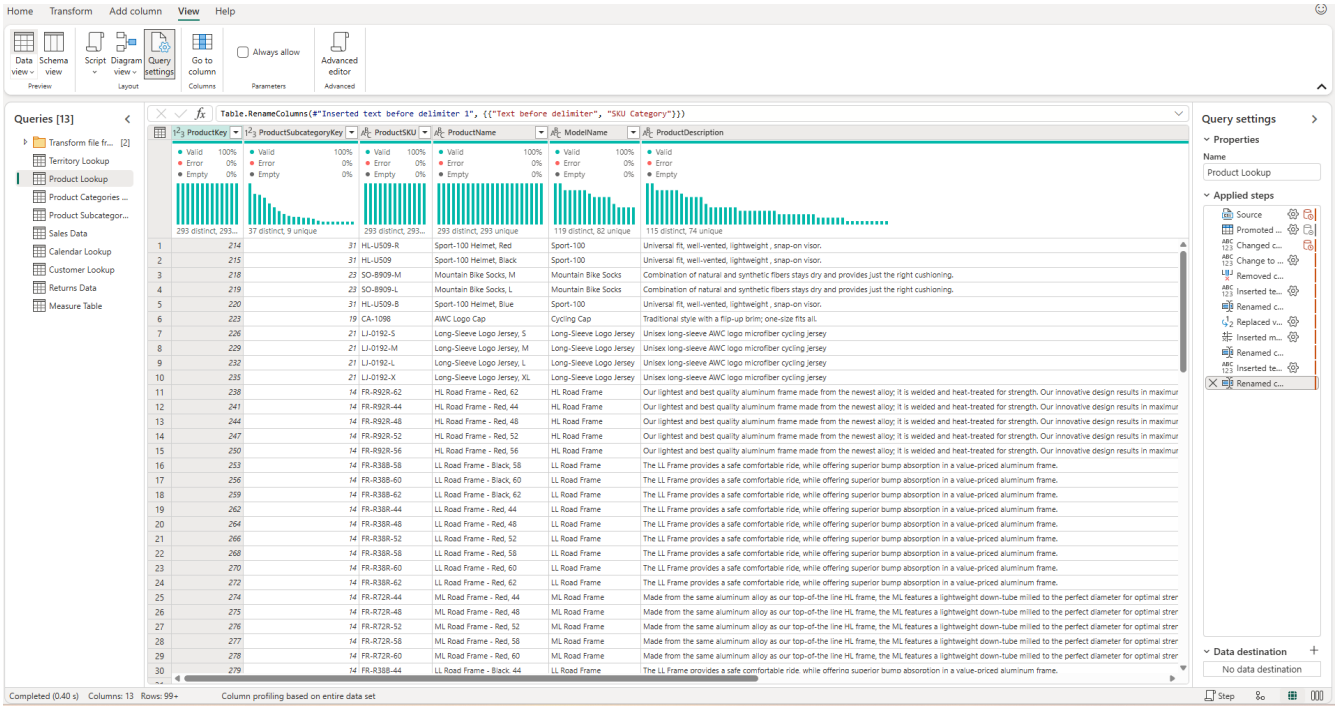


Figure 1. Product Lookup in Power Query: Queries on the left, the data preview in the center, and Query settings / Applied steps on the right.

What I learned to read in this view

Column quality reports the percentages of Valid, Error, and Empty values. **Column distribution** shows value patterns and distinct/unique counts. To investigate minimum and maximum values, I select a numeric or date column and review its Column profile statistics.

Distinct means the number of different values. **Unique** means values that occur exactly once. These indicators help me investigate keys; repeated values in a category column are not necessarily errors.

Profiling scope and Applied Steps

The screenshot states **Column profiling based on entire data set**. A profile based only on a 1,000-row preview could miss issues outside that sample. The preview label **Rows: 99+** is not proof of the total row count.

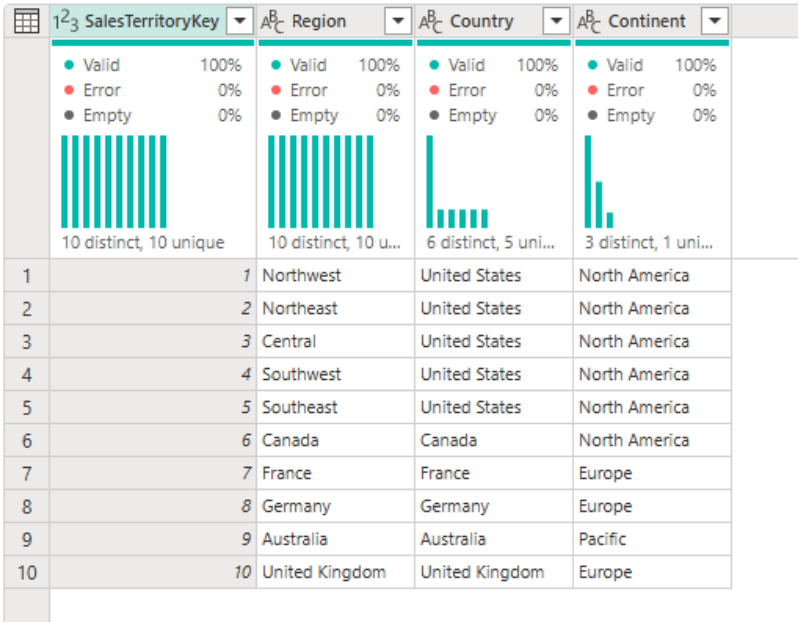
Applied Steps records the transformation sequence, including Source, Promoted Headers, Changed Type, added columns, and renaming. Selecting a step lets me review the data at that point; the formula bar shows the corresponding M expression.

Terminology: this is the data preview / Data view within Power Query, distinct from the model table view in Power BI Desktop. A 100% valid profile means no conversion errors were detected within the inspected scope; it does not prove every business rule is satisfied.

Preparing the Basic Lookup Tables

Territory Lookup | 10 rows, 4 columns

For this table, my recorded transformation was assigning appropriate data types. The key is a whole number, while the geographic labels are text.



	SalesTerritoryKey	Region	Country	Continent
	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%
	10 distinct, 10 unique	10 distinct, 10 u...	6 distinct, 5 uni...	3 distinct, 1 uni...
1	1	Northwest	United States	North America
2	2	Northeast	United States	North America
3	3	Central	United States	North America
4	4	Southwest	United States	North America
5	5	Southeast	United States	North America
6	6	Canada	Canada	North America
7	7	France	France	Europe
8	8	Germany	Germany	Europe
9	9	Australia	Australia	Pacific
10	10	United Kingdom	United Kingdom	Europe

Figure 2. SalesTerritoryKey has 10 distinct and 10 unique values. Region is also unique in this source. Country and Continent can repeat because a country may contain multiple regions.

Column	Appropriate type
SalesTerritoryKey	Whole Number
Region, Country, Continent	Text

Product Categories Lookup | 4 rows

Column	Appropriate type
ProductCategoryKey	Whole Number
CategoryName	Text

Product Subcategories Lookup | 37 rows

Column	Appropriate type
ProductSubcategoryKey, ProductCategoryKey	Whole Number
SubcategoryName	Text

I keep related key columns consistent in type. Categories and subcategories remain separate queries at this stage, ready for the relationships I will create during data modeling.

Product Lookup: Types and New Columns

The Product Lookup source contains 293 rows. I assigned data types and added columns to practice price calculations and text extraction.

Source column group	Appropriate type
ProductKey, ProductSubcategoryKey	Whole Number
ProductCost, ProductPrice	Fixed Decimal Number for monetary values
ProductSKU, ProductName, ModelName, ProductDescription, ProductColor, ProductSize, ProductStyle	Text; sizes may be codes rather than numbers

1. Creating Discount Price

Using Add Column > Custom Column, I calculated the price after a 10% discount. An illustrative M expression for this calculation is:

```
[ProductPrice] * (1 - 0.10)
```

The result is 90% of the original price and should use a monetary data type. This is a transformation exercise, not evidence that transactions received this discount. The documentation should follow the actual query column name, whether DiscountPrice or Discount Price.

2. Creating SKU Category

My intended rule was to take the first two characters of a SKU code. The source column is named **ProductSKU**; **SKU Type** in my original notes may refer to an intermediate column created during the exercise.

```
Text.Start([ProductSKU], 2)
```

This illustrates the two-character rule; it is not an export of the actual query. The screenshot shows a preceding step named **Inserted text before delimiter 1**, followed by renaming the output to **SKU Category**. Extracting before a hyphen can give the same result when all prefixes have two characters, but these rules are not universally equivalent.

What the Product screenshot confirms

The profiles support checks of completeness and value distribution. Applied Steps also includes remove, replace, and rename operations. I have not inferred their full details because the labels are truncated and the complete M query is unavailable.

What I learned: Power Query columns are calculated during transformation and refresh. They differ from DAX measures, which evaluate within the filter context when a report is used.

Sales Data: Combining Three Years

I combined three yearly sales files into one table. The underlying operation is **append**: stacking records with matching columns rather than adding attributes through a merge.

Source	Rows
Sales Data 2020	2,630
Sales Data 2021	23,935
Sales Data 2022	29,481
Combined Sales Data	56,046

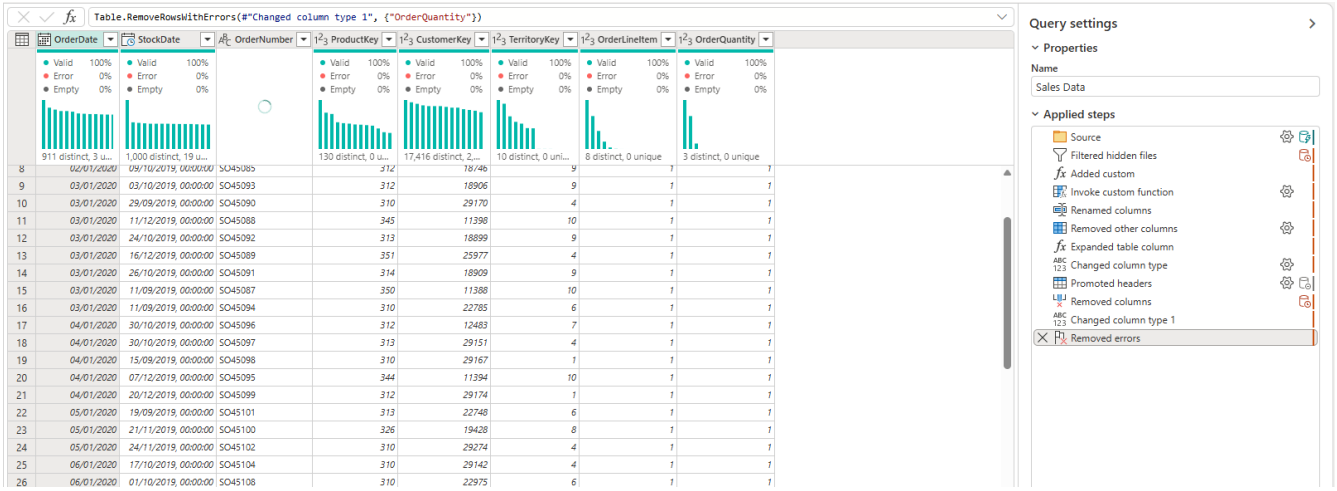


Figure 3. Sales Data and the Applied Steps sequence in the supplied screenshot.

Append and the Combine Files workflow

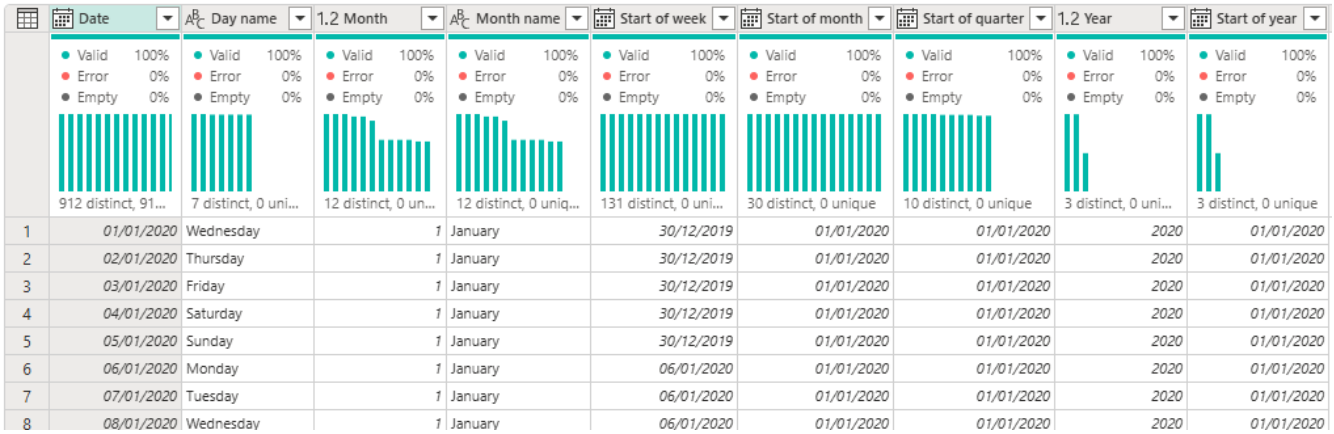
My learning notes refer to Append Queries. The screenshot includes Source, Filtered hidden files, Invoke custom function, and Expanded table column, indicating a **folder-based Combine Files** workflow. Both approaches can combine rows; the folder workflow also uses helper queries to transform files consistently.

Column	Appropriate type
OrderDate, StockDate	Date if time is not needed
OrderNumber	Text
ProductKey, CustomerKey, TerritoryKey, OrderLineItem, OrderQuantity	Whole Number

StockDate currently displays Date/Time values at 00:00:00. Date is more suitable if analysis uses only the date. The selected step runs Table.RemoveRowsWithErrors on OrderQuantity. This step alone does not prove records were removed: compare counts before and after, and investigate errors before discarding rows.

Calendar Lookup: Adding Date Attributes

The original CSV has one Date column and 912 rows. Through **Add Column > Date**, I created calendar attributes to support later time-based analysis.



	Date	Day name	1.2 Month	Month name	Start of week	Start of month	Start of quarter	1.2 Year	Start of year
	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%
	912 distinct, 91...	7 distinct, 0 uni...	12 distinct, 0 un...	12 distinct, 0 uniq...	131 distinct, 0 uni...	30 distinct, 0 unique	10 distinct, 0 unique	3 distinct, 0 uni...	3 distinct, 0 unique
1	01/01/2020	Wednesday	1	January	30/12/2019	01/01/2020	01/01/2020	2020	01/01/2020
2	02/01/2020	Thursday	1	January	30/12/2019	01/01/2020	01/01/2020	2020	01/01/2020
3	03/01/2020	Friday	1	January	30/12/2019	01/01/2020	01/01/2020	2020	01/01/2020
4	04/01/2020	Saturday	1	January	30/12/2019	01/01/2020	01/01/2020	2020	01/01/2020
5	05/01/2020	Sunday	1	January	30/12/2019	01/01/2020	01/01/2020	2020	01/01/2020
6	06/01/2020	Monday	1	January	06/01/2020	01/01/2020	01/01/2020	2020	01/01/2020
7	07/01/2020	Tuesday	1	January	06/01/2020	01/01/2020	01/01/2020	2020	01/01/2020
8	08/01/2020	Wednesday	1	January	06/01/2020	01/01/2020	01/01/2020	2020	01/01/2020

Figure 4. Calendar Lookup after adding date attributes.

Output column	Meaning	Appropriate type
Date	Source date	Date
Day name	Name of the day	Text
Month	Month number, 1-12	Whole Number
Month name	Name of the month	Text
Start of week	First date of the week	Date
Start of month	First date of the month	Date
Start of quarter	First date of the quarter	Date
Year	Calendar year	Whole Number
Start of year	First date of the year	Date

Checks prompted by the screenshot

Month and Year still show the **1.2** icon, indicating Decimal Number. Because their values are integers, I should change both to **Whole Number**. Date and the period-start columns already display date icons.

For January 1, 2020, Start of week is December 30, 2019, consistent with a Monday-start week. A week can begin in the previous year; this is not automatically a data error.

What I learned: day/month names are text labels, whereas period starts are dates. Locale affects the labels. Numeric columns can later help sort time labels correctly in the model.

Customer Lookup and Returns Data

Customer Lookup: checking the key

I checked CustomerKey for repeated values before using it as a lookup key. The supplied CSV has **18,154 rows** but **18,152 distinct CustomerKey values**: two excess occurrences relative to the distinct-key count.

This does not establish that every attribute in the repeated-key records is identical. Before removing duplicates by CustomerKey, I need to inspect conflicting attributes and decide which record to retain. The final query output is unavailable, so the 18,184 count in my initial notes remains unconfirmed.

Customer Lookup: creating Domain Name

I extracted the part after @ in EmailAddress using Add Column > Extract > Text After Delimiter. For a valid email format, an illustrative expression is:

```
Text.AfterDelimiter([EmailAddress], "@")
```

Domain Name uses Text. Empty email values or values without @ need checking so the extraction does not introduce new errors.

Customer source columns	Appropriate type
CustomerKey, TotalChildren	Whole Number
BirthDate	Date
AnnualIncome	Fixed Decimal Number after cleaning currency formatting
Prefix, FirstName, LastName, MaritalStatus, Gender, EmailAddress, EducationLevel, Occupation, HomeOwner	Text; the source HomeOwner field uses Y/N codes

Returns Data | 1,809 rows

My recorded transformation for Returns Data was changing data types. I chose types based on each column's meaning rather than relying only on automatic detection.

Column	Appropriate type
ReturnDate	Date
TerritoryKey, ProductKey, ReturnQuantity	Whole Number

Repeated keys are expected in a Returns transaction table. The unique-key rule for a lookup table should not be applied indiscriminately to transaction data.

Closing the Data Preparation Stage

I learned that data cleaning is more than deleting rows. I need to understand the structure, choose appropriate types, and explain why each transformation is necessary.

Learning summary

Profiling: quality indicators and distributions identify issues to investigate.
Types: keys, quantities, labels, money, and dates need different treatment.
Append: three years of sales become one analysis table.
Derived columns: calendar attributes, discount price, SKU category, and email domain enrich the data.
Applied Steps: the sequence provides a record I can review and revise.

Checks before Close & Apply

Check	Status in this document
Core CSV record counts	Calculated directly from 10 source files
Combined Sales count	56,046; reconciles to three yearly files
Final Customer Lookup count	Requires reconciliation with the final query
Repeated CustomerKey values	Detected in source; handling decision needs recording
Month and Year	Whole Number recommended
StockDate	Date recommended if time is unnecessary
SKU Category	Confirm first two characters vs before delimiter
Sales Remove Errors step	Compare record counts before and after
Complete M code and final column types	Not audited from full query definitions

Evidence and sources

This document draws on my supplied learning notes, 10 core CSVs, one additional practice CSV, and four Power Query screenshots: Product Lookup, Territory Lookup, Sales Data, and Calendar Lookup. Record counts exclude headers. Screenshots show the state captured at that moment and do not replace a full final-query audit.

Maven Analytics provided the course and AdventureWorks learning materials. M expressions in this journal are illustrative examples aligned with the described transformations, not exports of the original queries. This part focuses on connecting and shaping data; relationships, DAX, and dashboard development belong to later parts of the journal.

Project author: iffaaulia | Power BI Learning Journal